

HIGH ASSURANCE CLOUD COMPUTE AND STORAGE NATIONAL SECURITY AGENCY



Project Report

Prepared for NSA
March 26th, 2021

Technical Directors

Joshua Cline
jecline@radium.ncsc.mil
NSA

Eric Darko
eoagyen@nsa.gov
NSA

Jason Smith
Jnsmit8@nsa.gov
NSA

University Project Supervisor

Dr. Bhavani Thuraisingham
bhavani.thuraisingham@utdallas.edu

Team Members

Khalid Abusultan
Chetan Garikapati
Giakhanh Hoang
Monish Alur Gowdru
Travious Lewis

Table of Contents

Description	3
Motivation	3
Team Responsibilities	3
Deliverables	3
Budget	3
Plans	4
Schedule	4
Progress.....	4
SELinux	4
Exploits	5
Other Hardening Techniques	5
Cloud Data Sanitization.....	6
Computing and Software Resources	6
Literature Review	6
A Measurement Study on Linux Container Security: Attacks Countermeasures [4].....	6
Securing Docker Containers from Denial of Service (DoS) Attacks [1].....	7
Docker Ecosystem - Vulnerability Analysis [2]	7
Container Security: Issues, Challenges, and the Road Ahead [5].....	7
Securing A study on Container Vulnerability Exploit Detection [3]	8
Investigator Biographies	8
Khalid Abusultan.....	8
Chetan Garikapati.....	8
Giakhanh Hoang	9
Monish Alur Gowdru	9
Travious Lewis	9
Works Cited	10

Description

Container popularity has grown dramatically. Containers not only reduce the required resources for deployment, but they can be brought online or quickly deleted in a matter of seconds. However, they introduce unique new risks. Privilege escalation and cross container threats are among the security concerns, along with the access control and disposition of data. The government is interested in a new innovative, and comprehensive approach for securing containers and associated storage.

Motivation

Containers are the best platform for development of a project without impacting the existing project. We are interested in this project as it provides an opportunity to research on existing vulnerabilities to come up with solutions for securing containers with the help of user access to manage containers.

Team Responsibilities

- Reviewing literature on container security and vulnerabilities
- Learning to use containerization software such as Docker, container management/orchestration software such as OpenShift and Kubernetes
- Demonstrate and highlight vulnerabilities in a sandbox environment involving the tools mentioned above and other malware analysis tools as necessary
- Write weekly reports to sponsor and meet with them weekly for progress updates

Deliverables

- Proposal and Literature Review.
- Progress Report and Presentation.
- Final Report and Presentation.
- Documentation

Budget

We will use free trial versions of software such as OpenShift and if we need additional time or features, then we will need funding to continue to use that software. For now, there are no funds expected to be necessary for this project.

Plans

The group will meet weekly on Fridays at 10 am with our sponsors to check on the progress of the project. We may schedule more meetings for ourselves as necessary. We will use communication and time management to ensure project continues to progress on schedule.

Schedule

February 5: Bid due

February 12: Proposal due

February 19: Understanding the privileges that the container management software.

February 26: Baseline creating user components.

March 5: Hardening, SE-Linux for MAC.

March 12: Identifying Privilege Escalation.

March 25: Mid-term presentation submission due.

March 26: Mid-term presentation.

April 2: Create technical paper.

April 9: Sandbox experimentation

April 16: Continue sandbox experimentation.

April 23: Finish the technical paper.

April 30: Rehearsal for final presentation.

May 6: Final presentation submission due.

May 7: Final presentation & final report due.

Progress

SELinux

SELinux (Security-Enhanced Linux) is a Linux security module implementing stricter access control (such as Mandatory Access Control, or MAC) and more access control policies that administrators can use to secure their Linux installations. Developed by the NSA and Red Hat, this module would be good to have installed as a baseline for when we begin our experiments and tests for exploits.

We found that operating systems for servers such as CentOS already have SELinux installed and enabled by default when setup for the first time. This is convenient considering the administrator does not need

to go through the trouble of setting up/installing SELinux into their system. On other Linux systems where SELinux does not come installed by default such as Ubuntu, one of our team members has already found a case where the attempt to install SELinux causes the OS to crash POST-BIOS. We also learned that AWS does not support SELinux on their container platforms, only on dedicated servers such as a full CentOS installation.

Exploits

The focus was on privilege escalation attacks we worked on exploits involving vertical privilege escalation in cloud mapped storage volumes using username space remapping which maps an unprivileged user to docker volume which in turn modifies files on host system to gain root access. Another exploit involved taking advantage on trusted binaries, as the docker tries loading executables with predefined names without any checks, these binaries can be modified to create new users and to obtain root access, trusted binary exploits are more effective in windows systems.

Apart from this, we also worked on network attack vectors in which a compromised cloud image when deployed with network in bridge mode ARP spoofing can occur causing DoS attacks. This can also be generalized to create DNS spoofing and perform targeted exploits by involving Phishing. Such spoofing attacks can also be passive and can be used to perform reconnaissance on the targeted cloud infrastructure and perform more sophisticated attacks. Network attack vectors can also cause privilege escalation as some of the cloud instance sockets operate in privileged mode and attaching processes to such sockets or shaping the network traffic flow to these sockets in specific way to cause buffer overflow. Overflow attacks can also lead to disclosure of sensitive info using memory dumps.

One of the remedies is to perform system hardening to reduce privilege escalation risks and when using trusted libraries File Integrity Monitoring (FIM) solution must be used, if the storage volumes are huge then the location with libraries and logs must be monitored to avoid startup exploits.

Other Hardening Techniques

We found out that Linux hardening techniques can be used on Docker since it is based on Linux containers.

Some techniques that can be used are:

- Not running privileged on containers.
 - Using “--security-opt” for containers is encouraged.
 - Not every user needs to run on privileged, the daemon approach causes more vulnerabilities since malicious users can take advantage of running on privileged, and this causes security threats if poorly managed.
- Drop and observe dropped capabilities.
 - You can control different capabilities of processes in Docker by using:
 - #docker run --cap-add // To add the capabilities
 - #docker run --cap-drop // To drop the capabilities
 - Now you can drop all capabilities of Docker except net_bin_service, which lets us bind to a privileged process by running the following command

- `#docker run --cap-drop ALL --cap-add net_bind_service ghost /bin/bash`

Cloud Data Sanitization

We looked at few risks associated with not considering cloud-based data storage sanitization, which includes incomplete deletion leading to access for adversaries and data spillage (for example insecure storage unit that stores data in plain text).

We read the Department of Defense standards DoD Manual 5200.01 and DoD Manual 5220.22 and found that destroying the information identified for destruction to avoid reconstruction of information is set as the standards for cloud-based sanitization. Apart from this we looked at the standards followed by leading cloud providers for data sanitization.

- Google cloud uses six layers deep data security to protect the data and has a well-protected room for erasing and safe data destruction.
- Microsoft Azure and AWS follow guidelines provided by NIST 800-88 for data sanitization.

We learned about the techniques which can be used for protecting data which includes fine graining, maintain service availability during deletion, cloud computation, complete data destruction, error handling and acknowledge users after deletion. Also, we learned existing approaches for assured deletion which includes secure overwriting, disk scrubbing, and destroying encryption keys generated to access the data. Further research involves coming up with solution to assure that data deleted from the cloud is really gone.

Computing and Software Resources

We were able to successfully request and receive additional computing resources from UTD. We have a 24 thread / 96GB RAM Ubuntu host machine which is running three Red Hat Enterprise Linux (RHEL) VMs. We were also able to install an instance of Red Hat OpenShift on one of the RHEL VMs, so we now have that available to us for testing and experimentation.

Literature Review

[A Measurement Study on Linux Container Security: Attacks Countermeasures \[4\]](#)

The article evaluates the security of container mechanisms by manually executing the exploits on container platform and Linux platform. The article discusses the methodology of creating the exploit dataset containing 223 exploits against containers and development of two-dimensional attack taxonomy to classify collected exploits. Execution of 88 shortlisted exploits on Linux and container platform shows that there are many exploits which are still effective. In-depth analysis of privilege escalation attacks provides details of the effectiveness of kernel security mechanisms, container mechanism and CPU protection mechanisms on preventing privilege escalation against the container platform. The article includes extracting of a kernel privilege escalation attack model by analyzing the 11 exploits that could escape container boundary and gain root privilege and then proposes defense solution for all these

exploits. The paper aims at proving security of container mechanism depends on the security of the kernel, while the interdependence and mutual-influence relationship requires careful configurations to effectively defeat privilege escalation attacks.

[Securing Docker Containers from Denial of Service \(DoS\) Attacks \[1\]](#)

The article discusses some of the existing vulnerabilities or threats to containerization software and management, along with existing solutions typically built within the software itself to defend against an attack, and finally discusses possible methods to prevent Denial of Service (DoS) attacks. The existing vulnerabilities discussed include the Docker Daemon Attack Surface where a shared file space between the container and host can allow an attacker to gain access to the host by exploiting that shared space. Denial of Service attacks are also discussed, where the containers can communicate directly with the host kernel allowing an exploit where DoS attacks can be performed and privilege escalation can occur. Existing solutions to help secure the containers from attacks include basic security hardening that involve enabling software security layers such as SELinux. Another existing solution involves simply configuring Docker security policies. The article proposes limiting the memory of the container, reserving memory, or at least providing a default memory limit to prevent DoS attacks.

[Docker Ecosystem - Vulnerability Analysis \[2\]](#)

The paper discusses about docker containers and various vulnerabilities the docker containers face but can be generalized to all containers. The main security feature of a container is providing Isolation but in general deployments the containers share same network bridges and can lead to ARP poisoning attacks between other containers and host breaking the isolation. Using default host hardening techniques like using App Armor by default gives full access to host filesystems and network while the SELinux default profile puts all docker images under same domain so these restrictions protect host from containers but not containers from other containers. In the context of network security, the docker daemon is controlled via a Unix socket located at `/var/run/docker.sock` and this is owned by the “root” gaining access to this socket can run any container in privileged mode this in turn causes gaining root privileges on the host system as well. In case of TCP traffic any connection can obtain root if connections are not verified via TLS. When using the container as a VM to limit the administration tasks often leads to privilege creep and software developers prepacking the code in the containers also leads to increased attack surface as they reduce the configuration flexibility on deployment environments. Finally, when the container host os is not updated either for stability or mismanagement can easily result in breaking the container isolation by exploitation by kernel vulnerabilities. The paper also provides some guidelines but some of the remedies are deployment based and application specific.

[Container Security: Issues, Challenges, and the Road Ahead \[5\]](#)

This paper focuses on detailing four general use cases to describe the security issues that come with using containers, as well as available mechanisms to secure containers. The four use cases are: (I) protecting a container from applications running inside it, (II) inter-container protection, (III) protecting the host from containers, and (IV) protecting containers from a malicious or semi-honest host. This is followed by describing some container exploits, vulnerabilities, relevant standards, and discovery tools. The software-

based solutions for the first three use cases and hardware-based solution for the last use case are also discussed. The goal of the paper is to help researchers enhance their understanding of the different possible attacks and vulnerabilities of containers and to illustrate what their solutions provide, as well as highlight some open problems and future research directions.

Securing A study on Container Vulnerability Exploit Detection [3]

This paper focuses on a comprehensive analysis of 28 real world vulnerability attacks and test how effective container's vulnerability detections schemes are. Testing vulnerabilities using static and dynamic vulnerability attacks. The papers talked about utilizing various detection schemes to analyze container vulnerability detection. Such as K nearest neighbor scheme which uses two variables p and k to detect how many samples are identifies as anomalous. PCS + k-NN scheme which finds the presence noise (hundreds of different types of system calls). K-means clustering scheme in which clusters with small number of samples, based on a cluster size threshold is considered anomalous. And finally, SOM scheme which is a neural network that can reduce data dimensions and highlight similarities to find out anomalies. The paper found that static vulnerability scanning was insufficient since it was only able to detect 3 out of 28 vulnerabilities. Dynamic anomaly detection schemes can detect 22 vulnerabilities exploits with low positive rate. However, this is not the most accurate result. The study concluded that using a combination of both static and dynamic schemes the detection coverage was up to 86% (24 out of 28 vulnerabilities). [3] The authors of this study were able to provide a comprehensive review on using vulnerability detection schemes. The study recommends using both static and dynamic schemes to have the most accurate detections.

Investigator Biographies

Khalid Abusultan

Khalid Abusultan is currently an undergraduate senior starting a Master's degree in Computer Science with a focus in Information Assurance through the fast-track program. He is proficient in Python, C, and C++. He developed various web applications using Python frameworks such as Flask and Django. And used C and C++ to develop multithreaded applications in a Unix environment where he used Linux command line. He has done relevant coursework in Operating Systems, C/C++ Programming in Unix Environment, Computer Networks. He is currently taking coursework in Network Security, Web Programming, and Senior design Project. He is interested in cyber security and cloud security.

Chetan Garikapati

I have hands-on experience with enterprise application development as a full stack developer. I have done my undergraduate in computer science and gained knowledge on Computer Networks, Cryptography and Network Security and I am currently pursuing my Master's in

Cybersecurity with specialization in Information Assurance, where I have taken Advanced Computer Networks, Data Application Security and Privacy and Network Security. I also have a good understanding of regulatory compliance like HIPPA and FISMA and I also have developed Secure cloud applications with file sharing using Public Key Cryptography and local key management solutions. I also have hands-on with Linux systems (Debian based) and I am currently preparing for CompTIA Security+ certification.

Giakhanh Hoang

Giakhanh Hoang is a first year, graduate student studying computer science with a concentration in information assurance at The University of Texas at Dallas. He graduated in December 2020 with his Bachelor's of Science in Computer Science from The University of Texas at Dallas and matriculated to graduate school via the fast-track program. He chose to study in information assurance as he believes it will play a much greater role in the coming years with an increasingly connected world. He has taken classes on network security, system security, and malicious code analysis. He is currently taking classes for data and applications security, wireless networks, and information security this semester. He has experience programming in Java, C/C++, and Python. He also has experience working with UNIX-based operating systems, basic networking protocols, and virtual machines including Docker.

Monish Alur Gowdru

Monish Alur Gowdru is currently a second-year graduate student, pursuing Master's in Computer Science with a concentration in Information Assurance at The University of Texas at Dallas. He has completed Advanced Operating Systems, Network Security, Data and Application Security, and Information Security. He has experience programming in Java and C/C++. He is familiar with security tools Wireshark, Nmap, GDB, Ghidra, Burp Suite and Postman. He also has experience with full-stack development and UNIX-based operating systems. He has worked on finding vulnerabilities in IoT devices for PetCare, designing UNIX based file system and secure cloud platform for handling sensitive data. He is currently enrolled in Web Programming and Object-Oriented Analysis and Design.

Travious Lewis

Travious Lewis is currently a second-year graduate student, completing a Master's in Computer Science with a concentration in Information Assurance at The University of Texas at Dallas in Spring 2021. He graduated with a Bachelor of Science in Computer Engineering from Rose-Hulman Institute of Technology in 2019. He has been employed by IBM since 2019 working on projects that use a variety of technologies, including front and back-end web development, blockchain, cloud technologies such as Docker, Kubernetes, and Red Hat OpenShift, IoT, and IT automation. He has experience programming in JavaScript, Python, C/C++, Java, and Swift. He also is experienced with Bash and working in Unix environments, microcontrollers, mobile application development, and multithreaded applications. He has completed coursework in Advanced Computer Networks, Advanced Operating Systems, Information Security, and Network Security. He is currently taking courses in Wireless Networks and Data & Applications Security this semester.

Works Cited

- [1] J. Chelladhurai, S. A. P. Kumar, and P. R. Chelliah, "Securing Docker Containers from Denial of Service (DoS) Attacks," in *2016 IEEE International Conference on Services Computing (SCC)*, 2016, pp. 856–859.
- [2] A.Martin, S.Raponi, T.Combe and R.Di Pietro, "Docker ecosystem – Vulnerability Analysis" Nokia Bell Labs. <https://cseweb.ucsd.edu/~dstefan/cse227-fall18/papers/martin:docker.pdf>
- [3] O. Tunde-Onadele, J. He, T. Dai and X. Gu, "A Study on Container Vulnerability Exploit Detection," 2019 IEEE International Conference on Cloud Engineering (IC2E), Prague, Czech Republic, 2019, pp. 121-127, doi: 10.1109/IC2E.2019.00026.
- [4] Lin, Xin, et al. "A measurement study on Linux container security: Attacks and countermeasures." *Proceedings of the 34th Annual Computer Security Applications Conference*. 2018.
<https://csis.gmu.edu/ksun/publications/container-acsc18.pdf>
- [5] Sultan, Sari, Imtiaz Ahmad, and Tassos Dimitriou. "Container security: Issues, challenges, and the road ahead." *IEEE Access* 7 (2019): 52976-52996.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8693491>